TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

WEEKLY REPORT 2

Khảo sát không gian thiết kế shared L2 Cache
đánh giá bằng gem5

Team lead: PhD. Lê minh sơn
Mentor: Cao Văn Nam

Họ và tên SV: Choi Won Seok
MSSV: 24200032

Table of content

1. Progress Summary 1
2. DSE Automation Framework 1
2.1 Overview 1
2.2 Simulation Model and Configuration Parameters 2
2.3 DSE Script 2
2.4 Workload and Exploration Space 2
2.5 Summary 3
3. Simulation Data Processing Pipeline 3
4. Preliminary Results and Validation 4
5. Future Goals 6

Table of figures

Figure 1. difference in miss rate with different associativity. 5
Figure 2. Random policy performing worse in 64KiB 6

Weekly Report

1. Progress Summary

This week, I successfully developed and tested a Python script to automate design space exploration simulating different system configurations using the same workload and comparing their performance. Each simulation run stores its results in a separate directory. I also developed a second script to extract relevant performance metrics from the stats.txt files generated by each run and compile them into a single CSV file for further analysis.

The current system and workload are simplified test setups based on gem5's standard library, rather than the actual system planned for the project. Nevertheless, both scripts are working as intended and provide an initial implementation of the DSE automation and data pipeline. They can be extended and improved to support the project's actual system once it is ready, contributing to the overall project goals and requirements.

2. DSE Automation Framework

2.1 Overview

The objective of this work is to develop an automated simulation framework for exploring cache configurations in gem5. The framework currently consists of two Python scripts: model.py, which defines the simulation system, and sweep.py, which automates the execution of simulations across different cache configurations.

The current system is a preliminary model built primarily using gem5's standard library components. It consists of a single RISC-V MinorCPU core, an L1 instruction cache (icache), an L1 data cache (dcache), a shared L2 cache, and DDR3-1600 memory. The L1 and L2 cache classes are adapted from the examples in gem5's learning materials.

The automation script explores three L1 data cache parameters: cache capacity, associativity, and replacement policy. Although exploring the L1's capacity and associativity are not the primary focus of the project's eventual L2 design space exploration, sweeping these parameters helps validate the automation framework and provides a better understanding of cache behavior. The replacement-policy experiment also provides an initial comparison of how different policies affect cache miss rates.

Since the current model does not yet represent the final system specifications, the results are intended for preliminary analysis and validation rather than as definitive evidence for selecting configurations for the main system.

2.2 Simulation Model and Configuration Parameters

The simulation environment is defined in model.py. The system operates at a clock frequency of 1 GHz and uses a single RISC-V MinorCPU core. The cache hierarchy consists of separate L1 instruction and data caches connected to an L2 cache, with DDR3-1600 memory serving as the main memory.

The L1 instruction cache uses a fixed capacity of 16 KiB and associativity of 4, while the L2 cache is configured with a fixed capacity of 256 KiB. These parameters remain unchanged during the current experiments because the primary objective is to investigate the behavior of the L1 data cache.

Whereas the L1 data cache is configurable through command-line arguments. The parameters --l1d_size, --l1d_assoc, and --l1d_repl control its capacity, associativity, and replacement policy, respectively. This allows the same system model to be reused across multiple experiments without manually modifying the configuration for each run.

2.3 DSE Script

The sweep.py script automates the exploration of the selected cache parameters. It defines a set of candidate values for each parameter and iterates through their combinations using nested loops. For every combination, the script launches a gem5 simulation of model.py, passing the corresponding configuration values through command-line arguments.

To prevent simulation outputs from overwriting one another, each run is assigned a separate output directory. The directories follow a naming convention based on the L1 data cache capacity, associativity, and replacement policy, allowing the results to be identified and processed later.

This approach eliminates the need to launch each simulation manually and establishes a repeatable workflow for evaluating cache configurations. The resulting simulation statistics can then be collected by a subsequent data-processing stage for comparison and analysis.

2.4 Workload and Exploration Space

For the initial experiments, the system executes a synthetic workload that repeatedly traverses an integer array containing 8,192 elements and accumulates their values. Assuming each integer occupies 4 bytes, the array has a total size of 32 KiB. The traversal is repeated 50 times, resulting in 409,600 element accesses, excluding any additional memory accesses generated by instruction fetches, stack operations, and other runtime activity.

The exploration space consists of the following parameters:

- L1 data cache capacity: 4, 8, 16, 32, and 64 KiB.
- L1 data cache associativity: 1, 2, 4, and 8 ways.

- Replacement policy: LRU, FIFO, Random, and TreePLRU.

This configuration space produces 80 unique parameter combinations, but minus 5 since the TreePLRU policy doesn't work on a direct-mapped cache.

The workload provides a simple way to examine how cache capacity, associativity, and replacement policy influence memory-access behavior. Since the array occupies 32 KiB, cache capacity is expected to have a substantial effect on the results. However, the actual miss rate also depends on indexing, associativity, replacement behavior, and the memory-access sequence.

For example, a 32 KiB cache has enough capacity to hold the array, but this does not guarantee that all its elements can remain resident throughout execution. Cache mapping conflicts, and competing accesses can affect the result. Likewise, replacement policies may produce different outcomes when multiple memory blocks compete for the same cache sets. These effects will be evaluated using the simulation statistics.

2.5 Summary

The current framework establishes the basic infrastructure required for automated cache design space exploration: a configurable gem5 system model, an automated parameter sweep, and separate output directories for each experiment.

At this stage, the experiments focus on the L1 data cache rather than the shared L2 cache targeted by the main project. The current single-core model and basic workload are intended to validate the simulation and automation workflow before integrating the final system configuration.

Consequently, the results should not be used directly to select the final L2 configuration or replacement policy. Once the main system is available, the framework can be adapted to explore the shared L2 cache under the intended system configuration and workload. The generated statistics will then serve as inputs to the subsequent data-processing pipeline for structured comparison and reporting.

3. Simulation Data Processing Pipeline

After the automated simulations are completed, the results from each run need to be collected into a structured format for analysis. This is handled by parser.py, which iterates through the simulation output directories, reads the stats.txt file from each run, and extracts the relevant statistics into a single CSV file.

The parser currently generates 16 columns, organized into three categories: run information and configuration parameters, raw simulation statistics, and calculated performance metrics.

The first four columns identify each run and its configuration, including the output directory, cache capacity, associativity, and replacement policy. The next eight columns contain statistics extracted directly from gem5, including simulated time, executed instructions, CPU cycles, L1 data cache accesses and misses, L1 miss rate, average L1 miss latency, and L2 misses.

The final four columns contain calculated performance metrics that make it easier to compare configurations:

- CPI (Cycles Per Instruction): The average number of CPU cycles required per executed instruction.
- MPKI (Misses Per Kilo Instructions): The number of cache misses per 1,000 executed instructions, providing a measure of miss frequency relative to the amount of work performed.
- Average Miss Latency (cycles): The average L1 data cache miss latency converted from simulation ticks to CPU cycles.
- AMAT (Average Memory Access Time): The average time required per memory access, accounting for both cache hits and misses.

Together, these metrics provide a more consistent basis for comparing cache configurations than raw statistics alone. The resulting CSV also provides a structured input for subsequent analysis and visualization of the design space exploration results.

The parser is designed to be easily extended for future experiments. Statistics can be added or removed by modifying a list in the Python script, allowing the output CSV to be adapted to different analysis requirements with minimal changes to the implementation.

One current limitation is that the parser extracts configuration parameters by splitting the simulation output directory's name. This approach depends on a consistent naming convention and may become unreliable if future experiments use a different directory structure. To address this, a future implementation will extract configuration parameters directly from the `config.ini` file generated by gem5 in each run directory, making the parser more robust and less dependent on directory naming conventions.

4. Preliminary Results and Validation

After completing the simulations and collecting the relevant statistics into a single CSV file, the next step is to validate the simulation results and examine the observed cache behavior. This helps confirm that the automation workflow is functioning as intended while providing an initial understanding of how different cache configurations affect performance.

Across the completed runs, the number of executed instructions (simInsts) remained consistent, while L1 data cache accesses were approximately the same. This indicates

that the simulations executed a consistent workload, allowing the cache configurations to be compared meaningfully.

For cache capacities below 32 KiB, the L1 data cache miss counts remained relatively high, generally exceeding 20,000 misses. The results showed a gradual reduction in misses as cache capacity increased, while changes in associativity and replacement policy had comparatively little effect. This is consistent with the workload's 32 KiB array occupying a significant portion of the available cache capacity, limiting how much data can remain resident between traversals.

At 32 KiB and above, differences between associativity and replacement policies became more noticeable. The biggest difference can be seen in scenarios of cache size 32KiB where different value associativity can yield wildly different results.

	A	I
1	run	l1d_misses
47	l1d_32KiB_a1_FIFO	8836
48	l1d_32KiB_a1_LRU	8836
49	l1d_32KiB_a1_Random	8836
50	l1d_32KiB_a2_FIFO	11702
51	l1d_32KiB_a2_LRU	11679
52	l1d_32KiB_a2_Random	8948
53	l1d_32KiB_a2_TreePLRU	11679
54	l1d_32KiB_a4_FIFO	17630
55	l1d_32KiB_a4_LRU	17608
56	l1d_32KiB_a4_Random	9140
57	l1d_32KiB_a4_TreePLRU	17608
58	l1d_32KiB_a8_FIFO	3056
59	l1d_32KiB_a8_LRU	3279
60	l1d_32KiB_a8_Random	4037
61	l1d_32KiB_a8_TreePLRU	3279

Figure 1. difference in miss rate with different associativity.

The replacement-policy comparison showed that LRU, TreePLRU, and FIFO generally produced similar miss counts, while Random often performed worse. However, there were exceptions. For example, some configurations in the 32 KiB, showed substantially fewer misses with Random than with the other policies, while the 8-way configurations did not exhibit the same behavior. These variations demonstrate that replacement-policy performance depends on the interaction between the workload and cache configuration, rather than on the policy alone. Whereas in the configuration where cache size is 64KiB Random always performed worse.

1	run	l1d_misses
62	l1d_64KiB_a1_FIFO	2806
63	l1d_64KiB_a1_LRU	2806
64	l1d_64KiB_a1_Random	2806
65	l1d_64KiB_a2_FIFO	2783
66	l1d_64KiB_a2_LRU	2766
67	l1d_64KiB_a2_Random	2901
68	l1d_64KiB_a2_TreePLRU	2766
69	l1d_64KiB_a4_FIFO	2778
70	l1d_64KiB_a4_LRU	2766
71	l1d_64KiB_a4_Random	2984
72	l1d_64KiB_a4_TreePLRU	2766
73	l1d_64KiB_a8_FIFO	2775
74	l1d_64KiB_a8_LRU	2765
75	l1d_64KiB_a8_Random	3063
76	l1d_64KiB_a8_TreePLRU	2765

Figure 2. Random policy performing worse in 64KiB

Overall, the results provide an initial validation of the DSE workflow and demonstrate that the scripts can be used to investigate cache behavior across multiple configurations. However, these experiments use a simplified single-core system and a basic workload, so the results should not be treated as representative of the final shared L2 cache design. The primary achievement at this stage is establishing a working simulation and data-processing workflow that can be adapted to the main system when it becomes available.

5. Future Goals

Over the coming weeks, the goal is to improve the robustness of the existing scripts and coordinate more closely with the team to integrate the correct system configuration. This includes transitioning from the current single-core model to the intended four-core RISC-V system, allowing the team to begin evaluating more relevant performance statistics and conducting meaningful design space exploration of the shared L2 cache.